

PROGRAMAÇÃO ORIENTADA A OBJETOS - CHECKPOINT 1

Nome: _____

Nota: _____

RESPOSTAS

QUESTÃO 1	a	b	c	d	e
QUESTÃO 2	a	b	c	d	e
QUESTÕES 3-7	3	4	5	6	7
QUESTÕES 8-12	8	9	10	11	12

CENÁRIO: ELEVADOR

Os arquivos `Elevador.java` e `Main.java` estão no mesmo pacote. Considere exatamente o código abaixo.

Elevador.java	Main.java
<pre> 1 public class Elevador { 2 int andarAtual = 0; 3 boolean portaAberta = false; 4 5 void subir() { 6 andarAtual = andarAtual + 1; 7 } 8 void abrirPorta() { 9 portaAberta = true; 10 } 11 void fecharPorta() { 12 portaAberta = false; 13 } 14 int getAndarAtual() { 15 return andarAtual; 16 } 17 boolean getPortaAberta() { 18 return portaAberta; 19 } 20 }</pre>	<pre> 1 public class Main { 2 public static void main(String[] args) { 3 Elevador cabine = new Elevador(); 4 Elevador painel = cabine; 5 Elevador carga = new Elevador(); 6 7 painel.subir(); 8 cabine.abrirPorta(); 9 10 System.out.println(painel.getPortaAberta()); 11 System.out.println(painel.getAndarAtual()); 12 13 cabine.fecharPorta(); 14 15 System.out.println(cabine.getAndarAtual()); 16 System.out.println(cabine.getPortaAberta()); 17 18 carga.subir(); 19 carga.subir(); 20 carga.abrirPorta(); 21 22 System.out.println(carga.getAndarAtual()); 23 } 24 }</pre>

QUESTÕES

Questão 1 (25 pontos)

Faça o teste de mesa e indique o que será impresso em cada linha.

- | | |
|-------------|-------------|
| a) Linha 10 | d) Linha 16 |
| b) Linha 11 | e) Linha 22 |
| c) Linha 15 | |

Questão 2 (25 pontos)

Considere a evolução da classe e marque cada afirmação como V ou F.

Os getters e os métodos `abrirPorta()` e `fecharPorta()` permanecem inalterados.

Trechos alterados em Elevador.java
<pre> private int andarAtual; private boolean portaAberta; void subir() { if (portaAberta == false) { if (andarAtual < 3) { andarAtual = andarAtual + 1; } } }</pre>

- a) Em `Main`, a instrução `cabine.portaAberta = true;` compila.
- b) Métodos de `Elevador` não podem alterar `andarAtual` porque o campo é privado.

- c) Do térreo, após `abrirPorta()` e `subir()`, o elevador chega ao andar 1.
- d) Com a porta fechada e o andar em 2, chamar `subir()` leva ao andar 3.
- e) Do térreo, após `abrirPorta()`, `fecharPorta()` e `subir()`, o elevador chega ao andar 1.

Questão 3 (5 pontos)

O programa precisa guardar o andar e a situação da porta e executar as ações de subir, abrir e fechar. Qual organização representa melhor um elevador?

- A. Criar uma classe para o andar, outra para a porta e realizar as ações em `Main`.
- B. Manter os dados em variáveis de `Main` e passá-los a cada método externo.
- C. Criar uma classe apenas com o andar e a porta e programar todas as ações em `Main`.
- D. Manter os dois dados em `Main` e repetir ali todas as ações.
- E. Criar `Elevador` com o andar, a porta e os métodos que realizam essas ações.

Questão 4 (5 pontos)

Ao executar `Elevador cabine = new Elevador();`, o que cada parte da instrução representa?

- A. `cabine` é o objeto; `Elevador` guarda a referência usada por `new`.
- B. `new Elevador()` é a variável; `cabine` declara outra classe.
- C. `Elevador` é a classe; `cabine` guarda uma referência para o novo objeto.
- D. Os três nomes representam o mesmo papel e identificam diretamente o objeto.
- E. `Elevador` é a variável; `cabine` é a classe; `new` copia um objeto.

Questão 5 (5 pontos)

Quando `fecharPorta()` é executado, o valor de `portaAberta` muda. Qual alternativa descreve corretamente esses dois membros?

- A. Ambos são comportamentos porque podem ser acessados com ponto.
- B. `portaAberta` é estado; `fecharPorta()` é comportamento.
- C. O campo é uma classe e o método é um objeto.
- D. Ambos são estados porque pertencem ao mesmo objeto.
- E. `portaAberta` é comportamento; `fecharPorta()` é estado.

Questão 6 (5 pontos)

Queremos um método que receba um andar de destino inteiro e não retorne valor. Qual assinatura atende ao requisito?

- A. `void irPara(int destino)`
- B. `int irPara()`
- C. `void irPara(float portaAberta)`
- D. `int irPara(int destino)`
- E. `void irPara()`

Questão 7 (5 pontos)

O código compila? Se não, qual linha impede a compilação?

Trecho em `Elevador.java`

```
1 int andarAtual;
2 void mostrar() {
3     int destino;
4     System.out.println(andarAtual);
5     System.out.println(destino);
6 }
```

- A. Não compila: a variável local começa em 0, mas o campo precisa ser inicializado.
- B. Compila e as linhas 4 e 5 imprimem 0.
- C. Compila, mas a execução falha na linha 5 ao acessar `destino`.
- D. Não compila: a linha 5 tenta ler a variável local `destino` sem inicializá-la.
- E. Não compila: a linha 4 tenta ler o campo `andarAtual` sem inicializá-lo.

Questão 8 (5 pontos)

O elevador suporta no máximo 8 pessoas. Ao embarcar um grupo, a lotação não pode ultrapassar esse limite. Qual solução mantém essa regra em um único responsável?

- A. Um auxiliar de `Main` calcula a lotação e altera o campo.
- B. Um setter recebe a lotação calculada pelo código externo.
- C. `Elevador.embarcar(quantidade)` consulta sua lotação e preserva o limite.
- D. Cada trecho de `Main` consulta a lotação e decide se soma o grupo.
- E. `Main` verifica o grupo; outro método externo verifica o limite.

Questão 9 (5 pontos)

Considere o método e a chamada abaixo. O que acontece?

Trecho em `Elevador.java` e `Main.java`

```
1 int calcularDiferenca(int destino) {
2     return destino - andarAtual;
3 }
4 int diferenca = elevador.calcularDiferenca(3);
```

Considere que `andarAtual` vale 2 antes da chamada.

- A. `diferenca` recebe 3 e `andarAtual` passa a ser 3.
- B. `diferenca` recebe 1 e `andarAtual` continua em 2.
- C. O método não retorna valor porque não foi declarado com `void`.
- D. `diferenca` recebe 1 e `andarAtual` passa a ser 3.
- E. O código não compila porque um método com retorno `int` não pode receber parâmetro.

Questão 10 (5 pontos)

Dois elevadores são criados separadamente e começam no térreo com a porta fechada. Depois, apenas `x.abrirPorta()` é chamado. Qual afirmação está correta?

- A. Antes da chamada, `x == y` é `true`; depois, ambas as portas ficam abertas.
- B. Antes da chamada, os estados diferem; depois, somente a porta de `y` fica aberta.
- C. Antes da chamada, `x == y` é `true`; depois, somente a porta de `x` fica aberta.
- D. Antes da chamada, os estados são iguais; por isso, abrir `x` também abre `y`.
- E. Antes da chamada, os estados são iguais e `x == y` é `false`; depois, somente a porta de `x` fica aberta.

Questão 11 (5 pontos)

`getPortaAberta()` não possui modificador de acesso. Como `Main` e `Elevador` estão no mesmo pacote, qual afirmação está correta?

- A. Ele se torna privado automaticamente por não declarar `public`.
- B. Ele só pode ser chamado se `portaAberta` também for público.
- C. `Main` pode chamá-lo: o acesso de pacote permite uso no mesmo pacote.
- D. `Main` não pode chamá-lo porque o método não é `public`.
- E. Ele altera o pacote de `Main` durante a execução.

Questão 12 (5 pontos)

A porta só pode ser aberta no térreo. Analise a alteração abaixo. Qual diagnóstico está correto?

Trecho em `Elevador.java`

```
1 private boolean portaAberta;
2 public void definirPorta(boolean aberta) {
3     portaAberta = aberta;
4 }
```

- A. Receber um `boolean` preserva automaticamente todas as regras.
- B. O acesso direto fechou, mas o método ainda aceita qualquer mudança.
- C. Um método público não pode alterar um campo privado.
- D. Nem os métodos da própria classe podem alterar o campo.
- E. O campo `private` garante sozinho que toda mudança seja válida.